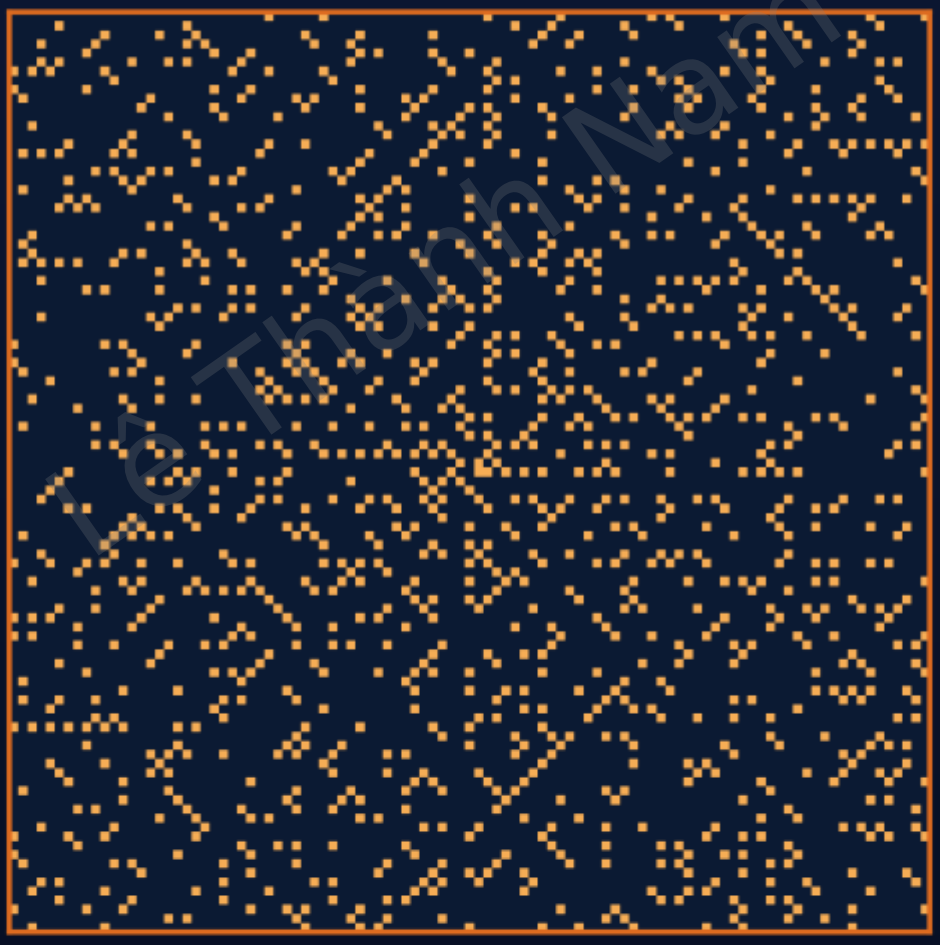
BỘ SÁCH BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

TUYỂN TẬP BÀI TẬP TOÁN TRÊN CODEFORCES

TẬP 2

SỐ NGUYÊN TỐ VÀ ƯỚC



Ảnh nền: xoắn ốc Ulam, ô sáng là số nguyên tố

Từ cách ngây thơ đến lời giải tối ưu · Chứng minh chi tiết · Mã nguồn C++ đã kiểm thử

Tổng hợp và biên soạn

LÊ THÀNH NAM

BỘ SÁCH BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

TUYỂN TẬP BÀI TẬP TOÁN TRÊN CODEFORCES

TẬP 2

SỐ NGUYÊN TỐ VÀ ƯỚC

Lê Thành Nam

Tổng hợp và biên soạn

LÊ THÀNH NAM

18 bài chọn lọc từ Codeforces, rating 900 đến 2000

Năm 2026

MỤC LỤC

Giới thiệu 5
Cuốn sách này dành cho ai 5
Cách giải 5
Subtask và các tầng lời giải 6
Sử dụng sách 6
Quy ước về chương trình 6
Ràng buộc và độ phức tạp 7
Bố cục sách 7
Chương 1. Số nguyên tố và sàng Eratosthenes 8
Kiến thức cần nhớ 8
Các bài trong chương 9
Bài 1. Số gần nguyên tố 10
Bài 2. Tô màu trang sức 17
Bài 3. Số T-nguyên tố 25
Bài 4. Tiền thuế 31
Chương 2. Cấu trúc ước và phân tích thừa số 38
Kiến thức cần nhớ 38
Các bài trong chương 38
Bài 5. Ước lẻ 39
Bài 6. Hai ước lớn nhất 46
Bài 7. Nhân và khai căn 53
Bài 8. Phép chia 59
Chương 3. Sàng ước nguyên tố nhỏ nhất và duyệt bội 65
Kiến thức cần nhớ 65
Các bài trong chương 65
Bài 9. Người mẫu 66
Bài 10. Máy va chạm 74
Bài 11. Cặp học viên 81

Lê Thành Nam

Bài 12. Chia mảng 88
Bài 13. Tổng ước 93
Bài 14. Khôi phục mảng 99
Chương 4. Giai thừa và công thức Legendre 105
Kiến thức cần nhớ 105
Các bài trong chương 105
Bài 15. Chữ số 0 tận cùng 106
Bài 16. Số nguyên tố và phép nhân 113
Chương 5. Hàm Möbius và bao hàm loại trừ 118
Kiến thức cần nhớ 118
Các bài trong chương 118
Bài 17. Dãy đặc biệt 119
Bài 18. Dãy con nguyên tố cùng nhau 127

Lê Thành Nam

Giới thiệu

Cuốn sách này dành cho ai

Sách gồm 18 bài về **số nguyên tố và ước** chọn từ kho đề Codeforces, xếp thành năm chương theo mạch kiến thức: số nguyên tố và sàng Eratosthenes; cấu trúc ước và phân tích thừa số; sàng ước nguyên tố nhỏ nhất và duyệt bội; giai thừa và công thức Legendre; hàm Möbius. Độ khó Codeforces trải từ 900 đến 2000: bài dễ nhất dành cho học sinh mới làm quen với số học trong lập trình, bài khó nhất ngang tầm đề thi học sinh giỏi cấp tỉnh và vòng loại quốc gia.

Mỗi bài gồm hai phần. **Đề bài** được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc và ví dụ của đề gốc, kèm **bảng subtask** do sách bổ sung. **Hướng dẫn giải** không đưa ngay lời giải tốt nhất mà đi qua nhiều **tầng**: lời giải ngây thơ, các tầng cải tiến, rồi mới đến lời giải tối ưu. Mỗi tầng đều có chương trình chạy được, có thời gian chạy đo thật và có hình minh họa.

> **Về cách viết đề.** Đề trong sách không phải bản dịch từng câu. Mỗi đề được viết lại theo cách hiểu tự nhiên của người Việt, bỏ những chi tiết kể chuyện không ảnh hưởng đến lời giải. Mọi điều kiện, giới hạn và ví dụ được giữ nguyên. Mỗi bài ghi mã bài và đường dẫn gốc để học sinh mở đề và nộp bài.

Cách giải

Hướng dẫn giải của mỗi bài chia thành các phần có dải màu riêng. Trong mỗi phần, lời giải đi từng bước nhỏ (Bước 1, Bước 2, ...), và hầu hết các bước đều có hình minh họa.

Phần	Nội dung
HIỂU ĐỀ	Vẽ ra các trường hợp để hiểu chính xác yêu cầu; nêu các nhận xét đầu tiên
NGÂY THƠ	Tầng 1: ý tưởng tự nhiên nhất, sơ đồ khối, chương trình vét cạn và độ phức tạp
BẾ TẮC	Thời gian chạy đo thật của tầng 1, chỉ ra nguyên nhân chậm, kết thúc bằng một câu hỏi then chốt
CẢI TIẾN	Các tầng trung gian: mỗi tầng sửa đúng một điểm yếu của tầng trước, có chương trình và subtask vượt qua được
LÓE SÁNG	Tính chất toán học quyết định dẫn tới lời giải tối ưu, có chứng minh và hình minh họa
TỐI ƯU	Chương trình tối ưu, độ phức tạp và bảng so sánh tất cả các tầng trên mọi subtask

Phần	Nội dung
LƯU Ý	Những chỗ dễ sai và phần mở rộng

Thời gian chạy trong sách được đo thật bằng trình biên dịch g++ 13 (tùy chọn -O2) trên máy chủ Linux, tính cả thời gian đọc dữ liệu. Trên máy chấm khác các con số có thể khác đôi chút, nhưng tỉ lệ giữa các tầng là như nhau.

Subtask và các tầng lời giải

Đề thi học sinh giỏi trong nước thường chia test thành nhiều **subtask** (nhóm test) với ràng buộc tăng dần. Vì vậy mỗi bài trong sách có một bảng subtask: subtask đầu dành cho lời giải ngây thơ, các subtask giữa dành cho các tầng cải tiến, subtask cuối là ràng buộc gốc của Codeforces. Với những bài mà đề gốc có ràng buộc nhỏ, sách **mở rộng** ràng buộc ở subtask cuối để lời giải tối ưu thật sự cần thiết.

Cuối mỗi bài có bảng so sánh: mỗi tầng chạy trên test lớn nhất của từng subtask, ghi thời gian thật và số điểm đạt được. Khi luyện thi, học sinh nên viết **ngay** tầng ngây thơ để lấy điểm subtask nhỏ và để có chương trình đối chiếu, rồi mới nâng cấp dần.

Sử dụng sách

Với mỗi bài, học sinh nên đọc đề, tự nghĩ ít nhất 20 đến 30 phút rồi mới đọc hướng dẫn. Khi đọc hướng dẫn, nên dừng sau mỗi khung **Câu hỏi then chốt** và tự trả lời trước khi đọc tiếp.

Mọi chương trình trong sách đã được chạy với ví dụ của đề và đối chiếu chéo với nhau trên 60 test ngẫu nhiên mỗi bài; các bài có nhiều đáp án được kiểm tra bằng chương trình chấm riêng. Chương trình tối ưu đã được đo trên test lớn nhất theo ràng buộc.

Quy ước về chương trình

- Viết bằng C++14, chỉ dùng các thư viện chuẩn cụ thể (iostream, cstdio, vector, algorithm), **không dùng** tệp bits/stdc++.h để biên dịch được trên phần mềm chấm Themis.
- Viết đầy đủ, không viết tắt: không macro, không typedef, mỗi câu lệnh một dòng, mọi khối lệnh có dấu ngoặc nhọn. Chương trình không chứa chú thích; mọi giải thích nằm trong phần hướng dẫn.
- Tên biến và tên hàm đơn giản, gần với công thức: **laNguyenTo**, **luyThua**, **spf**, **dem**, **kq**.

- Đọc ghi tệp **có điều kiện**: chương trình chỉ chuyển sang tệp CFxxxx.INP và CFxxxx.OUT khi tệp dữ liệu vào tồn tại. Nhờ vậy cùng một chương trình chạy được trên Themis và khi nộp lên Codeforces (đọc từ bàn phím).

Ràng buộc và độ phức tạp

Ràng buộc quyết định độ phức tạp được phép. Bảng dưới đây là "la bàn" cho mọi bài trong sách, với giả định máy chấm thực hiện khoảng 10^8 phép tính đơn giản mỗi giây.

Ràng buộc n	Độ phức tạp tối đa	Kĩ thuật thường gặp
≤ 20	$O(2^n \cdot n)$	vét cạn tập con
≤ 5000	$O(n^2)$	duyệt cặp, quy hoạch động hai chiều
$\leq 10^5$ đến 10^6	$O(n\sqrt{n})$, $O(n\log n)$	thử chia, sàng spf, duyệt bội
$\leq 10^7$	$O(n\log\log n)$	sàng Eratosthenes, sàng đếm
$\leq 10^{12}$	$O(\sqrt{n})$	thử chia tới căn bậc hai
$\leq 10^{18}$	$O(\log n)$	lũy thừa nhanh, Legendre

Bố cục sách

Chương	Chủ đề	Bài (mã Codeforces)
1	Số nguyên tố và sàng Eratosthenes	26A, 776B, 230B, 735D
2	Cấu trúc ước và phân tích thừa số	1475A, 1916B, 1062B, 1444A
3	Sàng ước nguyên tố nhỏ nhất và duyệt bội	1350B, 154B, 1771C, 1920C, 1512G, 1176D
4	Giai thừa và công thức Legendre	1114C, 1228C
5	Hàm Möbius và bao hàm loại trừ	900D, 803F

Lê Thành Nam

Chương 1. Số nguyên tố và sàng Eratosthenes

Trọng tâm chương. Mọi bài trong chương xoay quanh câu hỏi "số này có phải số nguyên tố không?", hỏi cho **rất nhiều số** cùng lúc. Kĩ thuật chung là **chuyển từ kiểm tra từng số sang tính trước cho cả dãy**: thay hàm laNguyenTo gọi đi gọi lại bằng một lần sàng Eratosthenes, rồi tra bảng trong $O(1)$. Chương này cũng rèn thói quen đọc ràng buộc để chọn giữa $O(\sqrt{n})$ cho mỗi số và $O(n\text{loglog}n)$ cho cả dãy.

Kiến thức cần nhớ

- **Định nghĩa.** Số nguyên tố là số tự nhiên lớn hơn 1 chỉ có đúng hai ước dương là 1 và chính nó. Số 1 không phải số nguyên tố; 2 là số nguyên tố chẵn duy nhất.
- **Kiểm tra nguyên tố trong $O(\sqrt{n})$.** Nếu n là hợp số thì n có một ước d với $2 \le d \le \sqrt{n}$. Vì vậy chỉ cần thử chia n cho các d thỏa $d \cdot d \le n$. Nên viết điều kiện vòng lặp là d * d <= n (dùng kiểu long long) thay cho d <= sqrt(n) để tránh sai số số thực.
- **Sàng Eratosthenes.** Đánh dấu mọi số từ 2 đến N; với mỗi i chưa bị gạch, gạch các bội $i^2, i^2 + i, \ldots$. Độ phức tạp $O(N\text{loglog}N)$, bộ nhớ $O(N)$.
- **Biến thể của sàng.** Thay vì chỉ đánh dấu đúng hoặc sai, có thể cộng dồn thông tin vào các bội: đếm số ước nguyên tố phân biệt, lưu ước nguyên tố nhỏ nhất.
- **Tính chất hay dùng.** Số có đúng 3 ước khi và chỉ khi nó là bình phương của một số nguyên tố. Mọi số nguyên lớn hơn 1 đều có ít nhất một ước nguyên tố.



Hình 0.1. Sàng đến 50: ô cam là số nguyên tố, ô trắng ghi ước nguyên tố nhỏ nhất

Các bài trong chương

Bài	Tên bài	Mã Codeforces	Độ khó	Số tầng
1	Số gần nguyên tố (Almost Prime)	26A	900	3
2	Tô màu trang sức (Sherlock and his girlfriend)	776B	1200	3
3	Số T-nguyên tố (T-primes)	230B	1300	3
4	Tiền thuế (Taxes)	735D	1600	2

Bài 1. Số gần nguyên tố

Codeforces 26A - Almost Prime Độ khó: 900

Đề gốc: https://codeforces.com/problemset/problem/26/A

Đề bài

Một số nguyên dương được gọi là **gần nguyên tố** nếu nó có **đúng hai** ước nguyên tố phân biệt. Chẳng hạn $6 = 2 \cdot 3$, $18 = 2 \cdot 3^2$ và $24 = 2^3 \cdot 3$ là các số gần nguyên tố; còn 4, 8, 9 (chỉ có một ước nguyên tố) và $42 = 2 \cdot 3 \cdot 7$ (có ba ước nguyên tố) thì không.

Cho số nguyên n, hãy đếm số lượng số gần nguyên tố trong đoạn từ 1 đến n.

Dữ liệu vào: Một dòng chứa số nguyên n ($1 \le n \le 3000$).

Kết quả: Một số nguyên là số lượng số gần nguyên tố trong đoạn $[1, n]$.

Ví dụ

Dữ liệu vào	Kết quả
10	2
21	8

Giải thích ví dụ. Với $n = 10$ có hai số gần nguyên tố là 6 và 10. Với $n = 21$ có tám số: 6, 10, 12, 14, 15, 18, 20, 21.

Subtask

Các subtask do sách bổ sung để luyện từng mức thuật toán; subtask chứa ràng buộc gốc được ghi chú, các subtask lớn hơn là mở rộng. Giới hạn thời gian 2 giây cho mọi subtask.

Subtask	Ràng buộc	Điểm
ST1	$n \le 3000$ (ràng buộc gốc)	30%
ST2	$n \le 10^5$	30%
ST3	$n \le 10^7$	40%

Hướng dẫn giải

HIỂU ĐỀ Hiểu đúng đề bài

Với mỗi số x, ta quan tâm đến **tập các ước nguyên tố khác nhau** của x, không quan tâm số mũ. Ví dụ $12 = 2^2 \cdot 3$ có tập ước nguyên tố $\{2,3\}$ (hai phần tử) nên 12 là số gần nguyên tố, dù 2 xuất hiện hai lần.



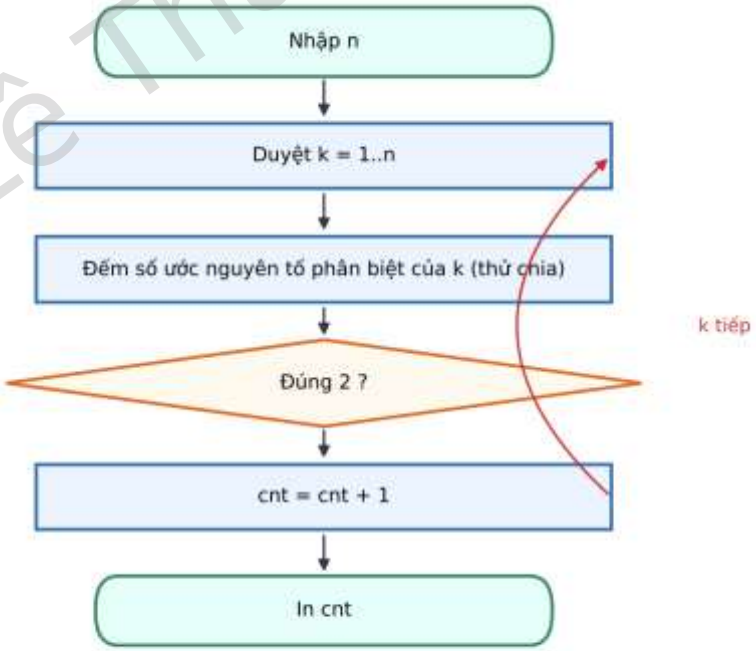
Hình 1.1. Các số gần nguyên tố đến 20 (ô cam)

- Số 1 không có ước nguyên tố nào; mỗi số nguyên tố có đúng một ước nguyên tố. Các số này không bao giờ được đếm.
- Lũy thừa của một số nguyên tố (4, 8, 9, 16, 27, ...) chỉ có **một** ước nguyên tố; $30 = 2 \cdot 3 \cdot 5$ có **ba**. Cả hai loại đều không được đếm.

NGÂY THƠ Ý tưởng ngây thơ: thử mọi p ≤ x và kiểm tra nguyên tố

Bước 1. Ý tưởng

Cách nghĩ trực tiếp nhất: với mỗi x từ 1 đến n, **thử mọi p từ 2 đến x**; nếu p chia hết x và p là số nguyên tố thì tăng biến đếm. Để kiểm tra p nguyên tố, thử mọi ước d từ 2 đến $p - 1$.



Hình 1.2. Sơ đồ khối của tầng 1

Lê Thành Nam

Bước 2. Cài đặt

Tầng 1: vét cạn

```
#include <iostream>
#include <cstdio>
using namespace std;

bool laNguyenTo(long long p) {
    if (p < 2) {
        return false;
    }
    for (long long d = 2; d < p; d++) {
        if (p % d == 0) {
            return false;
        }
    }
    return true;
}

int main() {
    FILE *f = fopen("CF26A.INP", "r");
    if (f != NULL) {
        fclose(f);
        freopen("CF26A.INP", "r", stdin);
        freopen("CF26A.OUT", "w", stdout);
    }
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int n;
    cin >> n;
    int dem = 0;
    for (int x = 1; x <= n; x++) {
        int soUoc = 0;
        for (int p = 2; p <= x; p++) {
            if (x % p == 0 && laNguyenTo(p)) {
                soUoc++;
            }
        }
        if (soUoc == 2) {
            dem++;
        }
    }
    cout << dem << "\n";
    return 0;
}
```

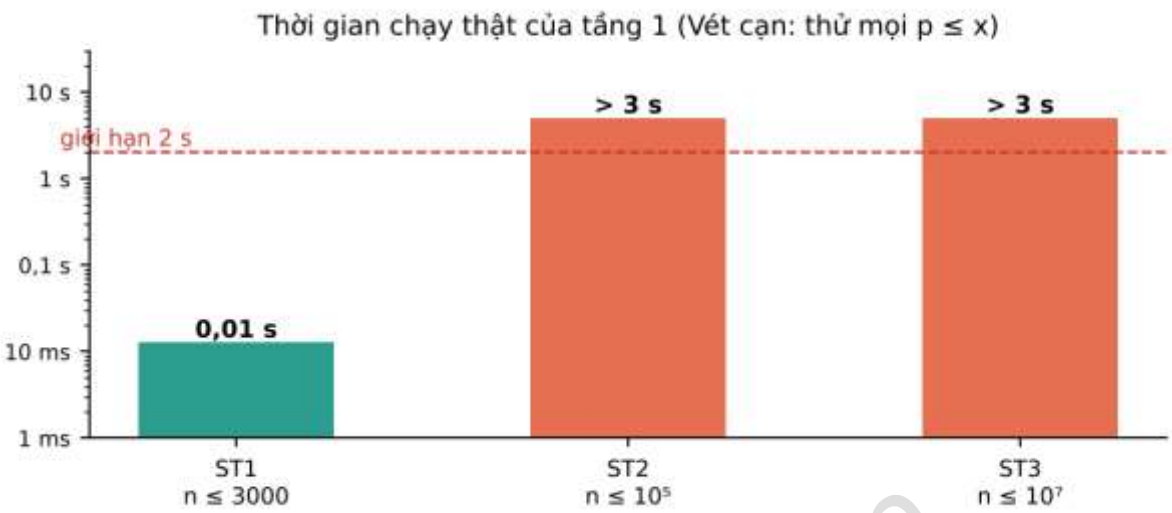
Lê Thành Nam

Bước 3. Đánh giá độ phức tạp

Vòng p chạy $x - 1$ lần cho mỗi x, nên riêng hai vòng lặp ngoài đã tốn $1 + 2 + \cdots + n \approx \frac{n^2}{2}$ phép chia. Mỗi lần p chia hết x, hàm laNguyenTo tốn thêm tới p phép chia; cộng theo mọi x, phần này cũng cỡ n^2. Tổng cộng $O(n^2)$: đủ cho ràng buộc gốc $n \leq 3000$ (khoảng $9 \cdot 10^6$ phép tính).

BẾ TẮC Bế tắc: vì sao vét cạn không dùng được khi n lớn

Bước 1. Đo thời gian thật



Hình 1.3. Thời gian chạy thật của tầng 1 trên test lớn nhất của từng subtask

Với $n = 3000$, tầng 1 chạy chỉ khoảng 10 ms. Nhưng mỗi khi n tăng 10 lần, thời gian tăng khoảng 100 lần: $n = 10^5$ đã quá giới hạn, còn $n = 10^7$ cần khoảng 10^{14} phép tính.

Bước 2. Tìm nguyên nhân

- **Thử thừa:** với $x = 30$ ta thử 29 giá trị p, nhưng chỉ 7 giá trị chia hết 30.
- **Lặp lại:** laNguyenTo(2) được gọi lại cho mọi số chẵn, laNguyenTo(3) cho mọi bội của 3, ... Kết quả luôn như nhau.

> **Câu hỏi then chốt.** Có cách nào tìm ra các ước nguyên tố của x mà **không phải thử mọi** $p \le x$, và không phải kiểm tra nguyên tố từng ước?

CẢI TIẾN 1 Cải tiến 1: phân tích thừa số bằng thử chia tới √x

Bước 1. Ý tưởng

Thử $d = 2,3,4,....$ Mỗi khi d chia hết x thì **chia x cho d tới khi không chia hết nữa** và tăng biến đếm. Khi đó d chắc chắn là số nguyên tố: mọi ước nhỏ hơn của d đã bị chia hết khỏi x trước đó, nên một hợp số d không thể chia hết phần còn lại. Không cần hàm laNguyenTo nữa.

Chỉ cần thử tới khi $d \cdot d$ vượt phần còn lại của x: nếu phần còn lại $y > 1$ không có ước nào không vượt quá $\sqrt{y}$ thì y là số nguyên tố, và nó là ước nguyên tố cuối cùng.

Lê Thành Nam

Tầng 2: thử chia tới căn bậc hai

```
#include <iostream>
#include <cstdio>
using namespace std;

int demUocNguyenTo(int x) {
    int dem = 0;
    for (int d = 2; d * d <= x; d++) {
        if (x % d == 0) {
            dem++;
            while (x % d == 0) {
                x = x / d;
            }
        }
    }
    if (x > 1) {
        dem++;
    }
    return dem;
}

int main() {
    FILE *f = fopen("CF26A.INP", "r");
    if (f != NULL) {
        fclose(f);
        freopen("CF26A.INP", "r", stdin);
        freopen("CF26A.OUT", "w", stdout);
    }
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int n;
    cin >> n;
    int dem = 0;
    for (int x = 1; x <= n; x++) {
        if (demUocNguyenTo(x) == 2) {
            dem++;
        }
    }
    cout << dem << "\n";
    return 0;
}
```

Lê Thành Nam

Bước 2. Kết quả

Mỗi số tốn không quá $\sqrt{x}$ phép chia, tổng $O(n\sqrt{n})$. Tầng 2 qua ST1 và ST2, nhưng $n = 10^7$ cần khoảng $10^7 \cdot 3 \cdot 10^3 = 3 \cdot 10^{10}$ phép chia, vẫn quá giới hạn.

> **Câu hỏi then chốt.** Hai số 12 và 24 có chung ước 2, 3 nhưng tầng 2 vẫn tìm lại từ đầu cho từng số. Có thể **tính trước** thông tin ước cho mọi số cùng lúc không?

LÓE SÁNG Lóe sáng: đảo chiều câu hỏi

Bước 1. Từ "x có những ước nào?" sang "p là ước của những số nào?"

Ta không cần biết các ước nguyên tố **cụ thể** của x, chỉ cần biết **có bao nhiêu**. Mà một số nguyên tố p là ước của đúng các số $p, 2p, 3p, \ldots$ Vậy với mỗi số nguyên tố p, cộng 1 vào ω của tất cả các bội của p. Cuối cùng $\omega[x]$ chính là số ước nguyên tố phân biệt của x.



Hình 1.4. Mảng ω sau khi xử lý từng số nguyên tố; ô cam có ω = 2

Bước 2. Nhận biết p nguyên tố ngay trong lúc sàng

Duyệt p tăng dần. Khi tới p, mọi số nguyên tố nhỏ hơn p đã cộng xong vào các bội của chúng. Nếu $\omega[p]$ vẫn bằng 0 thì không số nguyên tố nhỏ hơn nào chia hết p, tức p **là số nguyên tố**. Không cần mảng đánh dấu riêng.

TỐI ƯU Lời giải tối ưu

Bước 1. Cài đặt

Tầng 3: sàng đếm ước nguyên tố

```
#include <iostream>
#include <cstdio>
#include <vector>
using namespace std;

int main() {
    FILE *f = fopen("CF26A.INP", "r");
    if (f != NULL) {
        fclose(f);
        freopen("CF26A.INP", "r", stdin);
        freopen("CF26A.OUT", "w", stdout);
    }
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int n;
    cin >> n;
    vector<int> omega(n + 1, 0);
    for (int p = 2; p <= n; p++) {
        if (omega[p] == 0) {
            for (int j = p; j <= n; j += p) {
                omega[j]++;
            }
        }
    }
    int dem = 0;
    for (int i = 1; i <= n; i++) {
        if (omega[i] == 2) {
            dem++;
        }
    }
    cout << dem << "\n";
    return 0;
}
```

Lê Thành Nam

Bước 2. Độ phức tạp

Số phép cộng là $\frac{n}{2}+\frac{n}{3}+\frac{n}{5}+\frac{n}{7}+\cdots$ (tổng theo các số nguyên tố $p \le n$), bằng $O(n\text{loglog}n)$. Với $n = 10^7$, con số này chỉ khoảng $3 \cdot 10^7$.

Bước 3. So sánh các tầng

Tầng	Ý tưởng	Độ phức tạp	ST1	ST2	ST3	Điểm
1	Vét cạn: thử mọi p ≤ x	O(n²)	0,01 s	quá giờ	quá giờ	30
2	Thử chia tới √x	O(n√n)	2,6 ms	0,02 s	quá giờ	60
3	Sàng đếm ước nguyên tố	O(n log log n)	2,4 ms	3,2 ms	0,11 s	100

Thời gian đo thật (g++ -O2) trên test lớn nhất của từng subtask; giới hạn 2 giây. Ô đỏ: quá giờ, tràn bộ nhớ hoặc sai. Cột Điểm cộng phần trăm của các subtask mà tầng đó vượt qua.

LƯU Ý Những chỗ dễ sai

- Đếm nhầm ước có lặp: $12 = 2 \cdot 2 \cdot 3$ có **hai** ước nguyên tố phân biệt, không phải ba.
- Ở tầng 2, quên trường hợp phần còn lại $y > 1$ sau vòng lặp (ví dụ $14 = 2 \cdot 7$: vòng lặp dừng ở $d = 3$ và còn lại 7).
- Khi mở rộng n tới 10^7, mảng int tốn 40 MB; có thể dùng kiểu unsigned char (vì $\omega \le 8$) để giảm còn 10 MB.
- Mở rộng: cùng kĩ thuật "mỗi p cộng vào các bội" cho phép tính số ước, tổng ước, hàm Euler của mọi số không vượt quá n.

Lê Thành Nam